

AI Engineering Laboratory Manual

Lab Session 2

Building Local AI Agents with n8n and Llama

Student Edition

Lab Session 2: Building Local AI Agents with n8n and Llama

Context and lab intent

This lab is designed for final-year engineering students in an AI-focused program that emphasizes hands-on learning, project-based work, and industry-ready tool usage.[page:1] The second lab moves from API-based GenAI usage into local agent building, where students run n8n on their laptops and connect it to a locally hosted Llama model for private, reproducible AI workflows.[web:36][web:42]

Lab overview

- Lab block: Sessions 4-6
- Suggested duration: 3 hours per session, 9 hours total
- Theme: Local AI agents with n8n and Llama
- Runtime approach: fully local on Mac and Windows laptops
- Core tools: Docker Desktop, n8n, and a local Llama runtime such as Ollama or llama.cpp
- Pedagogy: setup, guided workflow building, debugging, and reflection

Learning objectives

By the end of this lab, students should be able to:

- Install Docker Desktop on Mac or Windows and run a containerized local development stack.
- Install and run n8n locally and access it through the browser.
- Install and run a local Llama-based model runtime on the same machine.
- Connect n8n to a local LLM endpoint and create a basic AI agent workflow.

- Use prompt templates, memory, tools, and simple routing logic inside n8n.
- Debug common setup and workflow issues on both macOS and Windows.
- Explain the privacy, latency, cost, and reliability differences between local and cloud AI.
- Document the architecture of a local agent system clearly.

Prerequisites

Students should have:

- A MacBook or Windows laptop.
- At least 16 GB RAM recommended; 8 GB may work for smaller models but with reduced performance.
- 20+ GB free disk space for Docker images and model files.
- Admin rights to install Docker Desktop.
- Stable internet for initial downloads.
- Basic familiarity with browser-based tools and terminal usage.

Software stack

Students must install:

1. Docker Desktop.
2. n8n running locally in Docker.
3. Ollama or llama.cpp for local model hosting.
4. A compatible Llama model downloaded locally.
5. A browser, code editor, and optional API testing tool.

Installation guide

For Mac students

1. Install Docker Desktop for Mac from Docker's official download page.[web:36]
2. Install Ollama using the official installer or command-line setup from the Ollama download page.[web:42]
3. Start Docker Desktop and verify it is running.
4. Open Terminal and prepare to run n8n in Docker.

For Windows students

1. Install Docker Desktop for Windows from Docker's official download page.[web:36]

2. Use Docker Desktop to run n8n in a container, or use the provided `docker run` workflow.[web:36]
3. Install Ollama for Windows if supported in the lab setup, or use the local runtime path approved by the instructor.
4. Verify that Docker, WSL2 support, and the terminal are working properly before starting the lab.

Recommended Docker commands

Create persistent storage

```
docker volume create n8n_data
```

Run n8n locally

```
docker run -it --rm --name n8n -p 5678:5678 -e GENERIC_TIMEZONE="Asia/Kolkata" -e TZ="Asia/Kolkata" -e N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true -e N8N_RUNNERS_ENABLED=true -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

This setup is directly aligned with n8n's Docker guidance, which recommends Docker for most self-hosting needs and uses port 5678 with a persistent volume for the `.n8n` directory.[web:36]

Session structure

Session	Focus	Core outcome
---------	-------	--------------

---	---	---
-----	-----	-----

4	Local environment setup	Students install Docker, n8n, and a local Llama runtime
---	-------------------------	---

5	First local agent workflow	Students connect n8n to Llama and build a simple agent
---	----------------------------	--

6	Multi-step local automation	Students add tools, routing, memory, and testing
---	-----------------------------	--

Session 4

Local environment setup

Session objective

Students should get n8n and a local Llama runtime working on their own laptops and verify that they can access both services reliably.

Concept coverage

- Why local AI matters: privacy, latency, offline capability, and cost control

- Container basics: image, container, volume, port mapping
- Why n8n is useful for visual orchestration
- Why a local Llama model is useful for private AI workflows
- Minimum hardware and model-size tradeoffs

Instructor demonstration

The instructor should show the complete flow once on a Mac and once on Windows:

- Install Docker Desktop.
- Start Docker.
- Run n8n.
- Open `http://localhost:5678`.
- Start the local Llama runtime.
- Confirm the model responds to a simple prompt.

Exercise 4.1: Install Docker Desktop

Step-by-step instructions

1. Download Docker Desktop for the student's operating system.
2. Install it with default settings.
3. Open Docker Desktop and confirm it is running.
4. On Windows, ensure virtualization and WSL2 are enabled if required.
5. Record a screenshot of Docker running.

Expected outcome

Students should have Docker Desktop installed and operational.

Exercise 4.2: Run n8n locally

Step-by-step instructions

1. Open Terminal on Mac or PowerShell on Windows.
2. Create the persistent volume using `docker volume create n8n_data`.
3. Run the provided `docker run` command.
4. Wait for the n8n startup log.
5. Open `http://localhost:5678` in a browser.
6. Create the initial n8n owner account.
7. Save the login credentials securely for lab use.

Expected outcome

Students should see the n8n editor UI on their local machine.

Exercise 4.3: Install and verify local Llama

Step-by-step instructions

1. Install Ollama or the instructor-approved local Llama runtime.
2. Download a small Llama model suitable for student laptops.
3. Test the runtime with one prompt from the terminal or GUI.
4. Confirm that the response is generated locally.
5. Document the runtime name, model name, and any errors encountered.

Expected outcome

Students should have a local model endpoint ready for workflow integration.

Exercise 4.4: Environment check worksheet

Task

Students complete a checklist covering Docker, n8n, and Llama runtime status.

Checklist

- Docker Desktop installed.
- Docker running.
- n8n available at localhost.
- Local model downloaded.
- Local model responds to prompts.
- Firewall or port issues resolved.

Expected outcome

Students should leave the session with a verified local AI development environment.

Deliverable for Session 4

- Screenshot of Docker and n8n.
- Screenshot or terminal log proving the local Llama runtime works.
- Completed environment checklist.

Session 5

First local agent workflow

Session objective

Students should connect n8n to the local Llama model and create their first visual AI workflow.

Concept coverage

- Workflow nodes and data flow
- Trigger, input, model call, output
- Prompt templating inside workflows
- Why agents differ from simple prompt calls
- Basic error handling in n8n

Instructor demonstration

The instructor should build a minimal workflow:

1. Manual Trigger.
2. Set Node for input text.
3. LLM Node connected to local Llama.
4. Output node to display the response.

Exercise 5.1: Build a simple chat workflow

Step-by-step instructions

1. Open n8n locally.
2. Create a new workflow named `Local Llama Chat`.
3. Add a Manual Trigger node.
4. Add a node that sends a prompt to the local Llama model.
5. Enter a system instruction such as: `You are a helpful tutor for final-year engineering students.`
6. Add a user prompt field.
7. Execute the workflow.
8. Verify the response in the output panel.

Expected outcome

Students should have a basic working AI chat workflow inside n8n.

Exercise 5.2: Prompt template design

Step-by-step instructions

1. Modify the workflow so the user prompt is dynamic.
2. Add placeholders for topic, audience, and output format.
3. Test the same workflow with three different prompts.

4. Compare output quality.
5. Record which placeholders improved consistency.

Expected outcome

Students should understand how workflow variables shape model behavior.

Exercise 5.3: Add structured output

Step-by-step instructions

1. Change the prompt so the model must return JSON.
2. Include fields such as title, summary, action_items, and difficulty_level.
3. Execute the workflow.
4. Validate whether the output is parseable JSON.
5. If needed, refine the prompt and rerun.

Expected outcome

Students should learn to use local models for machine-readable workflow outputs.

Exercise 5.4: Compare local and chat-based behavior

Step-by-step instructions

1. Ask the same question in the local model and in ChatGPT.
2. Compare style, completeness, speed, and confidence.
3. Write a short note on where local models are sufficient and where cloud models may still be useful.

Expected outcome

Students should understand tradeoffs between local and cloud AI use.

Deliverable for Session 5

- One working n8n workflow using local Llama.
- Screenshot of the workflow and output.
- One-page comparison note on local versus cloud behavior.

Session 6

Multi-step local automation

Session objective

Students should extend the simple workflow into a local AI agent that can take inputs, process them, and call tools or follow branching logic.

Concept coverage

- Agentic workflow design
- Routing based on task type
- Tool use in n8n
- Basic memory or state handling
- Retrieval from local text or files
- Evaluation and debugging practices

Instructor demonstration

The instructor should show one agentic workflow with:

- A user input trigger.
- A classifier step.
- A branch for summarization or extraction.
- A local LLM step.
- A final output formatter.

Exercise 6.1: Build a task router

Step-by-step instructions

1. Create a new workflow named `AI Task Router`.
2. Accept user input such as a paragraph or question.
3. Add a classification step that decides whether the input is a summary task, extraction task, or explanation task.
4. Use branching logic to route to the correct prompt.
5. Run three sample inputs.
6. Confirm that each input reaches the correct branch.

Expected outcome

Students should understand how agents can route work based on intent.

Exercise 6.2: Add a local knowledge source

Step-by-step instructions

1. Create a short local text file with notes or course content.
2. Load the content into the workflow using a file, text, or HTTP step.
3. Ask the model to answer a question using only the local notes.
4. Instruct the model to say when the answer is not present in the source.
5. Test with both answerable and unanswerable questions.

Expected outcome

Students should understand a basic retrieval-style pattern using local information.

Exercise 6.3: Add output verification

Step-by-step instructions

1. Add a validation step after the model response.
2. Check whether the output is complete, structured, and within the requested format.
3. If it fails, send it back through a correction prompt.
4. Compare first-pass and corrected outputs.

Expected outcome

Students should see that agent workflows often need a verification loop.

Exercise 6.4: Debugging lab

Step-by-step instructions

1. Intentionally break one part of the workflow: prompt, node connection, or model setting.
2. Observe the failure.
3. Restore the workflow.
4. Write down the error cause and fix.

Expected outcome

Students should build confidence in troubleshooting local AI workflows.

Deliverable for Session 6

- A multi-step n8n workflow with branching logic.
- One local knowledge-based workflow.
- A debugging note describing one problem and its fix.

Assessment plan

Component	Weightage
Environment setup and verification	20%
Basic local workflow	25%
Multi-step agent workflow	30%
Debugging and reflection	15%

| Documentation quality | 10% |

Evaluation rubric

| Criterion | Excellent | Good | Needs improvement |

| --- | --- | --- | --- |

| Local setup | Docker, n8n, and Llama all work correctly | Minor setup issues resolved with help | Setup incomplete or unstable |

| Workflow design | Clear, modular, and logically routed | Mostly correct with limited modularity | Workflow is hard to follow or broken |

| Agent behavior | Uses branching, prompts, and structured output well | Basic agent flow works | Only a simple one-step call works |

| Debugging skill | Diagnoses and fixes issues independently | Fixes issues with partial support | Unable to troubleshoot effectively |

| Documentation | Clear screenshots and explanations | Adequate but incomplete | Missing key evidence |

Expected cumulative outcomes

By the end of Sessions 4-6, students should be able to:

- Run a local AI stack on a personal laptop.
- Understand the relationship between Docker, n8n, and local LLM runtimes.
- Build and test local AI workflows visually.
- Route tasks and verify outputs in a workflow-based agent system.
- Explain why local AI is valuable for privacy-sensitive and cost-sensitive scenarios.

Suggested instructor notes

- Keep one model choice standard for the whole class to reduce support issues.
- For Windows users, provide a short pre-lab guide on Docker Desktop and virtualization.
- For Mac users, provide separate notes for Apple Silicon and Intel if needed.
- Use small models first so students can complete the workflow without waiting too long.
- Encourage students to save screenshots after every successful step.

Materials to provide students

- Docker installation guide for Mac and Windows.
- n8n starter workflow file.

- Local Llama installation guide.
- Sample text files for retrieval and extraction.
- Troubleshooting checklist.
- Lab report template.

Submission checklist

Students must submit:

- Installation screenshots.
- Working workflow screenshots.
- One exported n8n workflow file.
- Debugging notes.
- Short reflection on local AI advantages and limitations.

Expected final lab outcome

At the end of this second lab block, students should be able to stand up a private local AI environment and build practical n8n-based AI agents that use a local Llama model for task routing, structured output, and workflow automation. This supports strong conceptual understanding while preparing students for the next lab on end-to-end application integration.

